



Praktikum Pemrograman Mobile

Pertemuan 3:

Dynamic Lists with Lazy Layouts

Disusun oleh :

Azis Amirulbahar, S.P.d., M.T.I.

Program Studi Informatika
Fakultas Teknik
Universitas Jenderal Soedirman
2026



DAFTAR ISI

DAFTAR ISI	2
PENDAHULUAN	3
A. Pengantar	3
B. Tujuan Praktikum	3
C. Alat dan Bahan	4
D. Dasar Teori.....	4
TAHAPAN PRAKTIKUM	5
A. Membuat Package Baru	5
B. Membuat Data Class Category.....	5
C. Membuat Data Class Product.....	7
D. Membuat Data Dummy.....	7
E. Membuat Layout untuk Menampilkan Daftar Produk.....	11
F. Membuat Layout untuk Menampilkan Kategori Produk.....	14
G. Pratinjau Function dengan anotasi @Preview	16
H. Implementasi Lazy List	17
I. Membuat Activity Baru	23
J. Modifikasi HomeActivity.....	26

PENDAHULUAN

A. Pengantar

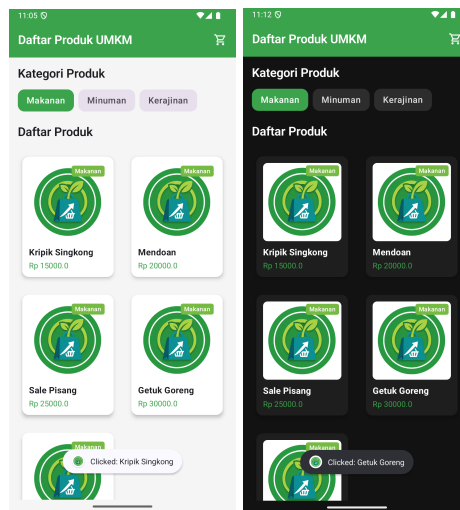
Pada pertemuan ke-3, kita akan mempelajari konsep *Dynamic Lists* (Daftar yg bersifat Dinamis) dan *Lazy Layouts*. Misalnya, aplikasi menampilkan 1000 daftar produk, menuliskannya kode dalam kode untuk setiap produk secara manual (*hardcode*) satu per satu tentu sangat tidak efisien dan akan membuat memori perangkat menjadi penuh. Solusinya adalah menggunakan *Lazy Layouts*. Kata "*Lazy*" (malas) berarti sistem hanya akan "menggambar" atau merender elemen antarmuka (UI) yang sedang terlihat di layar saja. Jika pengguna men-*scroll* layar, elemen yang sudah tidak terlihat akan dihapus dari memori dan digunakan ulang untuk elemen baru yang muncul. Sehingga, kinerja aplikasi berjalan lancar dan ringan.

"Modul pada pertemuan ke-3 dibuat lebih ringkas daripada pertemuan – pertemuan sebelumnya. Peserta diharapkan dapat secara mandiri untuk melakukan implementasi suatu tampilan dari kode yang dibuat dengan mengadopsi pertemuan sebelumnya, tanpa petunjuk/panduan baris kode seperti pada pertemuan 1 dan 2. Terdapat pula implementasi komponen yang sama dengan pertemuan – pertemuan sebelumnya, sehingga penjelasan kode tidak perlu disertakan pada pertemuan ini dan peserta diharapkan secara mandiri untuk dapat mengimplementasikannya."

B. Tujuan Praktikum

1. Memahami perbedaan antara layout standar (*Column/Row*) dengan *Lazy Layouts* (*LazyColumn/LazyRow*).
2. Menerapkan *LazyColumn* untuk menampilkan daftar data dalam jumlah banyak secara efisien.
3. Menggunakan *Data Class* di Kotlin untuk menstrukturkan data list.
4. Membuat desain *Item List* kustom menggunakan komponen *Compose* (seperti *Card*, *Row*, dan *Image/Icon*).

Hasil akhir praktikum adalah sebagai berikut:



Gambar 1. Hasil Akhir Praktikum

C. Alat dan Bahan

1. Perangkat keras (PC/Laptop) dengan spesifikasi minimum sesuai standar Android Studio.
2. Android Studio (versi terbaru yang sudah mendukung Jetpack Compose secara bawaan).
3. Emulator Android (AVD) atau perangkat fisik Android yang terhubung melalui *debugging* (USB/Wi-Fi).

D. Dasar Teori

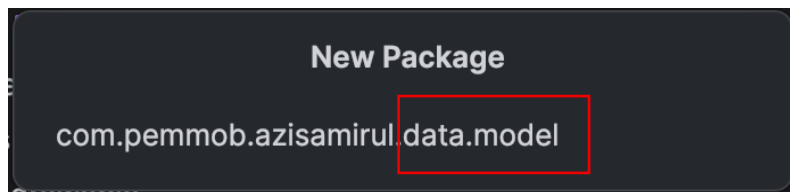
1. **LazyColumn (Vertikal) / LazyRow (Horizontal)**: Hanya melakukan *render* item yang terlihat di layar pengguna. Saat pengguna menggulir layar/*scrolling*, item yang keluar dari layar akan “didaur ulang”, dan item baru akan dimuat. Sehingga, sangat efisien untuk daftar data yang panjang.
2. **Data Class** adalah kelas khusus di Kotlin yang dirancang secara spesifik untuk merepresentasikan data. Hanya dengan menambahkan kata kunci `data` di depan deklarasi `class`, Kotlin akan secara otomatis membuatkan fungsi-fungsi pendukung di belakang layar seperti *Setter Getter* yang terdapat di bahasa pemrograman Java.

TAHAPAN PRAKTIKUM

Pada bagian ini, peserta diminta untuk mengikuti tahapan – tahapan untuk menghasilkan aplikasi sederhana seperti pada gambar 1 di halaman sebelumnya.

A. Membuat Package Baru

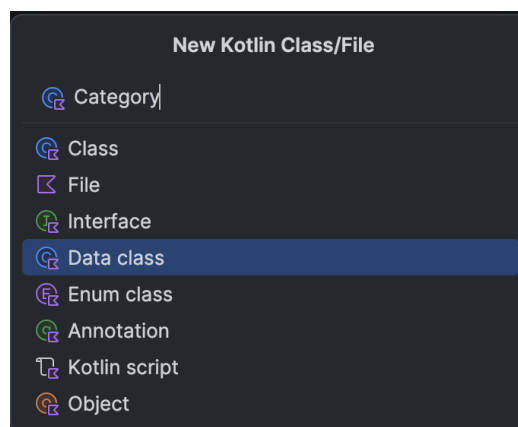
Buatlah *package* baru yang Bernama **.data.model** seperti gambar berikut (menyesuaikan dengan nama *package* masing – masing peserta praktikum):



Gambar 2. Membuat Package Baru dengan Nama .data.model

B. Membuat Data Class Category

1. Buatlah Kotlin File baru dengan nama: Category yang berjenis Data Class



Gambar 3. Membuat Data Class Category

2. Buka `Category.kt` dan tuliskan kode berikut:

```
data class Category(  
    val id: Int,  
    val name: String,  
    val description: String?,  
    val products_count: Int?  
)
```

Gambar 4. Implementasi Data Class Category

Kode di atas mendefinisikan sebuah struktur data bernama `Category` menggunakan fitur `data class` pada Kotlin. Penggunaan kata kunci `val` pada seluruh parameternya menandakan bahwa nilai-nilai tersebut bersifat *immutable* atau tidak dapat diubah lagi setelah objek pertama kali dibuat. Selain itu, terdapat penambahan tanda tanya (?) pada tipe data `String?` dan `Int?` yang merupakan implementasi dari fitur *Null Safety* di Kotlin. Tanda tersebut secara eksplisit mengindikasikan bahwa properti tersebut bersifat opsional dan diizinkan untuk tidak memiliki nilai sama sekali (`null`) tanpa menyebabkan aplikasi *error* (`NullPointerException`).

Nama Properti	Tipe Data	Status Null (Nullability)	Penjelasan
id	Int	Wajib Ada (Tidak boleh null)	Menyimpan nomor identitas unik dari kategori berupa angka bulat (contoh: 1, 2, 3). Karena tidak ada tanda ?, data wajib diisi.
name	String	Wajib Ada (Tidak boleh null)	Menyimpan nama kategori berupa teks (contoh: "Elektronik", "Pakaian"). Data ini juga wajib diisi.
description	String?	Opsional (Boleh null)	Menyimpan teks penjelasan panjang tentang kategori. Adanya ? berarti kolom ini boleh dibiarkan kosong jika kategori tersebut belum memiliki deskripsi.

Nama Properti	Tipe Data	Status Null (Nullability)	Penjelasan
products_count	Int?	Optional (Boleh null)	Menyimpan angka jumlah produk yang ada di dalam kategori tersebut. Boleh bernilai null jika sistem belum menghitung atau datanya belum tersedia.

C. Membuat Data Class Product

Setelah membuat data Class Category, silakan dilanjutkan untuk mendeklarasikan Data Class Product, yang berisi variabel seperti pada gambar 5 di bawah ini.

```
data class Product(
    val id: Int,
    val category_id: Int,
    val category: Category?,
    val name: String,
    val description: String?,
    val price: Double,
    val stock: Int,
    val img:String
)
```

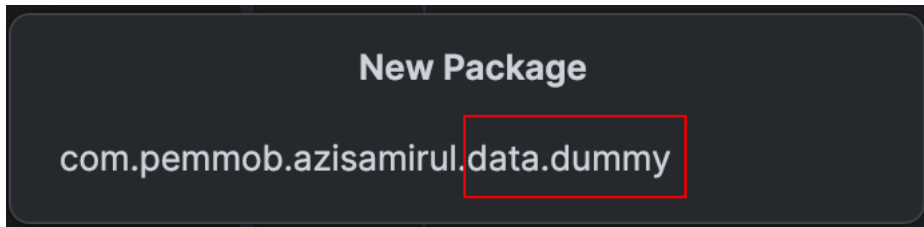
Gambar 5. Deklarasi Data Class Product

D. Membuat Data Dummy

Data Dummy adalah sekumpulan data fiktif atau data tiruan yang sengaja dibuat langsung di dalam kode program (*hardcoded*) untuk keperluan uji coba dan desain antarmuka. Sebelum aplikasi benar-benar terhubung ke *database* asli atau API (*Application Programming Interface*) untuk melihat bagaimana daftar list, teks, atau gambar akan dirender di layar. Pada Jetpack Compose, *data dummy* biasanya diimplementasikan dengan membuat sebuah fungsi yang mengembalikan koleksi (seperti List) berisi sekumpulan objek dari sebuah *Data Class*, sehingga *layout* seperti LazyColumn dapat langsung diuji secara visual.

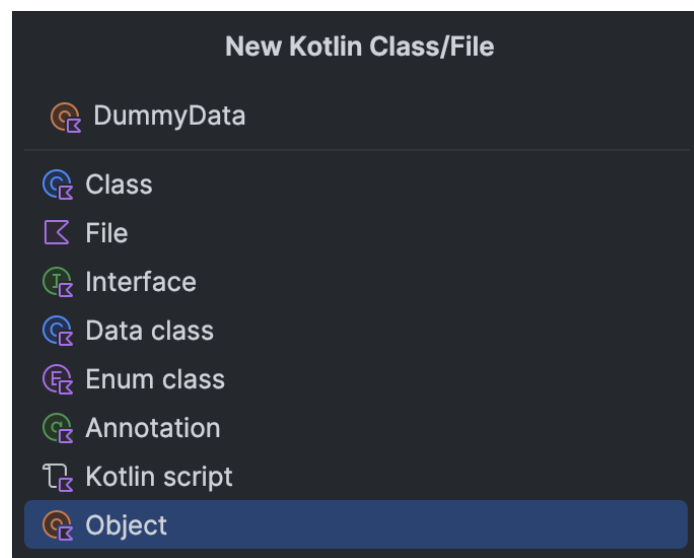
Pada tahapan ini, kita akan mempraktikkan untuk membuat data *dummy*. Silakan ikuti langkah – langkah berikut:

1. Membuat Package baru dengan nama **.data.dummy**



Gambar 6. Membuat Package Baru Bernama Dummy

2. Membuat Kotlin File Class Baru dengan Nama DummyData yang bertipe Object.



Gambar 7. Membuat File Baru Bertipe Object

Dalam Kotlin, kata kunci `object` digunakan secara khusus untuk mendeklarasikan sebuah *Singleton*, yaitu kelas yang secara otomatis hanya memiliki satu instance (objek) tunggal di dalam memori selama aplikasi berjalan, dan langsung diinisialisasi saat pertama kali diakses. Hal ini sangat berbeda dengan *Java*, di mana pembuatan *Singleton* mengharuskan pengembang menulis kode panjang secara manual, seperti membuat konstruktor berstatus *private* dan metode *static*.

Selain itu, untuk inisialisasi atau pembuatan objek biasa secara umum, perbedaan utamanya adalah Kotlin sepenuhnya menghilangkan penggunaan kata kunci `new` yang wajib ada di Java. Di Java, pembuatan objek ditulis seperti Mahasiswa `mhs = new Mahasiswa()`; , sedangkan di Kotlin pengembang cukup memanggil nama kelas layaknya memanggil sebuah fungsi biasa, misalnya `val mhs = Mahasiswa()` , sehingga penulisan kode menjadi jauh lebih ringkas.

3. Implementasi Object untuk DummyData Category

Silakan tuliskan kode berikut:

```
object DummyData {
    18 Usages
    val categories = listOf(
        Category(id = 1, name = "Makanan", description = "Aneka Makanan Lokal", products_count = 5),
        Category(id = 2, name = "Minuman", description = "Minuman Segar", products_count = 5),
        Category(id = 3, name = "Kerajinan", description = "Kerajinan Tangan", products_count = 5)
    )
}
```

Gambar 8. Dummy Data Category

4. Implementasi Object untuk DummyData Product

Silakan tuliskan kode berikut:

```
object DummyData {
    17 Usages
    val categories = listOf(
        Category(id = 1, name = "Makanan", description = "Aneka Makanan Lokal", products_count = 5),
        Category(id = 2, name = "Minuman", description = "Minuman Segar", products_count = 5),
        Category(id = 3, name = "Kerajinan", description = "Kerajinan Tangan", products_count = 5)
    )

    3 Usages
    val products = listOf(
        // Makanan
        Product(id = 1, category_id = 1, category = categories[0], name = "Kripik Singkong", description = "Kripik gurih", price = 15000.0, stock = 50, img = "dummy_product"),
        Product(id = 2, category_id = 1, category = categories[0], name = "Mendoan", description = "Mendoan asli Purbalingga", price = 20000.0, stock = 30, img = "dummy_product"),
        Product(id = 3, category_id = 1, category = categories[0], name = "Sate Pisang", description = "Sate pisang manis", price = 25000.0, stock = 20, img = "dummy_product"),
        Product(id = 4, category_id = 1, category = categories[0], name = "Batuk Goreng", description = "Batuk khas", price = 30000.0, stock = 40, img = "dummy_product"),
        Product(id = 5, category_id = 1, category = categories[0], name = "Nopis", description = "Nopis rasa coklat", price = 22000.0, stock = 60, img = "dummy_product"),

        // Minuman
        Product(id = 6, category_id = 2, category = categories[1], name = "Es Dawet", description = "Dawet seger", price = 10000.0, stock = 100, img = "dummy_product"),
        Product(id = 7, category_id = 2, category = categories[1], name = "Kopi Robusta", description = "Kopi bubuk", price = 45000.0, stock = 20, img = "dummy_product"),
        Product(id = 8, category_id = 2, category = categories[1], name = "Wedang Jahe", description = "Jahe lantan", price = 12000.0, stock = 50, img = "dummy_product"),
        Product(id = 9, category_id = 2, category = categories[1], name = "Teh Poci", description = "Teh melati", price = 15000.0, stock = 40, img = "dummy_product"),
        Product(id = 10, category_id = 2, category = categories[1], name = "Sirup Stroberi", description = "Sirup rasa", price = 35000.0, stock = 15, img = "dummy_product"),

        // Kerajinan
        Product(id = 11, category_id = 3, category = categories[2], name = "Batik Purbalingga", description = "Kain batik", price = 150000.0, stock = 10, img = "dummy_product"),
        Product(id = 12, category_id = 3, category = categories[2], name = "Sandal Bandol", description = "Sandal awet", price = 40000.0, stock = 25, img = "dummy_product"),
        Product(id = 13, category_id = 3, category = categories[2], name = "Sapu Glagah", description = "Sapu lantai", price = 25000.0, stock = 100, img = "dummy_product"),
        Product(id = 14, category_id = 3, category = categories[2], name = "Gantungan Kunci", description = "Gantungan kayu", price = 5000.0, stock = 150, img = "dummy_product"),
        Product(id = 15, category_id = 3, category = categories[2], name = "Tas Rajut", description = "Tas wanita rajut", price = 85000.0, stock = 5, img = "dummy_product")
    )
}
```

Gambar 9. Implementasi Kode untuk Dummy Product

Penjelasan Kode:

Kode `DummyData` pada nomor 4 dan 5 mendefinisikan sebuah *Singleton* menggunakan kata kunci `object` dengan nama `DummyData`. Karena menggunakan `object`, `DummyData` akan langsung diinisialisasi sebagai satu-satunya *instance* di memori, sehingga pengembang dapat mengakses isinya secara langsung tanpa perlu membuat objek baru (misalnya dengan memanggil `DummyData.categories`). Di dalamnya terdapat dua properti berjenis `val`, yaitu `categories` dan `products`, yang diisi dengan koleksi data fiktif menggunakan fungsi `listOf`.

`List` adalah sebuah tipe koleksi/*collection* data yang berfungsi untuk menyimpan sekumpulan nilai atau objek dalam satu urutan tertentu secara terstruktur. Setiap elemen di dalam sebuah `List` memiliki nomor posisi berurutan yang disebut indeks, yang selalu dimulai dari angka nol (0). Secara bawaan, fungsi `listOf()` yang digunakan pada Kotlin akan menghasilkan `List` yang bersifat *read-only* (hanya-baca), yang berarti isi dan jumlah elemen di dalamnya tidak dapat ditambah, dikurangi, atau diubah setelah koleksi tersebut diinisialisasi.

Properti `categories` menyimpan daftar objek dari *data class* `Category`, sementara `products` menyimpan daftar objek `Product` yang mewakili produk-produk lokal khas daerah seperti "Mendoan", "Nopia", hingga "Sapu Glagah", di mana setiap produk tersebut secara otomatis ditautkan kembali ke kategori yang sesuai berdasarkan nomor indeks (contoh: `categories[0]` untuk menunjuk kategori "Makanan").

E. Membuat Layout untuk Menampilkan Daftar Produk

Pada bagian ini, kita akan membuat layout daftar produk. Silakan buat terlebih dahulu file .kt baru bernama `DaftarProductScreen.kt` yang diletakkan ke package `.ui.screen`

1. Masih berada di file `DaftarProductScreen.kt`, buatlah function baru seperti pada gambar di bawah ini:

```
@Composable
fun ProductItemCard(product: Product, onClick: () -> Unit) {
```

Gambar 10. Deklarasi Function ProductItemCard

Penjelasan Kode:

Kode di atas merupakan deklarasi sebuah fungsi bernama `ProductItemCard` menggunakan kata kunci `fun` pada Kotlin. Fungsi ini menerima dua buah parameter masukan. **Parameter pertama** adalah **product yang bertipe data Product**, berfungsi untuk membawa informasi produk yang akan ditampilkan pada antarmuka. Parameter kedua adalah **onClick bertipe () -> Unit**, yang merupakan implementasi dari konsep **Lambda** atau fungsi yang dikirimkan sebagai parameter (**Higher-Order Function**). Sintaks `() -> Unit` menandakan bahwa `onClick` akan menerima sebuah blok perintah atau aksi yang tidak memerlukan data masukan `()` dan tidak mengembalikan nilai kembalian (`Unit`). Pendekatan ini digunakan untuk "memicu" sebuah aksi khusus (seperti berpindah halaman atau menampilkan pesan) yang akan dieksekusi saat kartu produk tersebut diklik oleh pengguna.

2. Membuat Card di dalam ProductItemCard

Tulislah kode seperti pada gambar 11 berikut:

```
@Composable
fun ProductItemCard(product: Product, onClick: () -> Unit) {
    Card(
        modifier = Modifier
            .padding(all = 8.dp)
            .fillMaxWidth()
            .clickable { onClick() },
        elevation = CardDefaults.cardElevation(defaultElevation = 4.dp),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surface)
    ) {
    }
}
```

Gambar 11. Implementasi Card pada Function ProductItemCard

3. Membuat Column dan Box di dalam Card

Tuliskan kode seperti pada gambar 12 berikut yang terletak **di dalam card** setelah mengimplementasikan seperti pada gambar 11.

```
Column(modifier = Modifier.padding(all = 12.dp)) {
    val imageRes = if (product.img == "dummy_product") R.drawable.dummy_product else R.drawable.dummy_product
    Box(
        modifier = Modifier.fillMaxWidth()
    ) {
    }
}
```

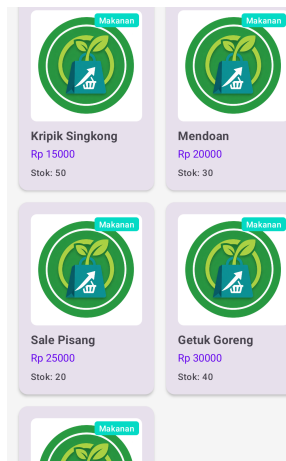
Gambar 12. Implementasi Column dan Box

Kode pada gambar 12 mendeklarasikan sebuah variabel *immutable* bernama `imageRes` untuk menyimpan referensi ID dari sebuah sumber gambar (*drawable*). Salah satu keunggulan dan ciri khas Kotlin yang diimplementasikan pada potongan kode di atas adalah digunakannya `if-else` sebagai sebuah *expression* (ekspresi) yang mampu langsung menghasilkan atau mengembalikan nilai.

Alih-alih membuat variabel kosong terlebih dahulu lalu mengisinya di dalam blok pengecekan seperti pada bahasa pemrograman Java, Kotlin memungkinkan untuk langsung menugaskan hasil evaluasi dari `if-else` ke dalam variabel `imageRes` dalam satu baris kode yang ringkas. Secara logika, kode tersebut memeriksa apakah teks pada properti `product.img` bernilai `"dummy_product"`, di mana pada contoh khusus ini, baik kondisi

terpenuhi (*if*) maupun tidak (*else*) tetap akan memberikan gambar cadangan yang sama, yaitu `R.drawable.dummy_product`.

Pada bagian ini, peserta secara mandiri untuk dapat melakukan import gambar produk yang akan digunakan seperti pada gambar 13 berikut dengan dimensi gambar adalah 1:1, peserta dibebaskan untuk berkreasi/ mencari gambar secara mandiri, dan silakan sesuaikan nama filenya pada saat import pemanggilan di fungsi *if* seperti pada gambar 12.



Gambar 13. Contoh Penggunaan Gambar Produk

4. Implementasi Image dan Box

Silakan tulis kode seperti pada gambar 14 berikut **di dalam Box**, melanjutkan dari hasil implementasi seperti pada gambar gambar 12:

```
Image(  
    painter = painterResource(id = imageRes),  
    contentDescription = product.name,  
    modifier = Modifier  
        .fillMaxWidth()  
        .aspectRatio(1f)  
        .clip(shape = RoundedCornerShape(size = 8.dp))  
        .background(color = androidx.compose.ui.graphics.Color.White),  
    contentScale = ContentScale.Fit  
)  
  
if (product.category != null) {  
    Box(  
        modifier = Modifier  
            .align(Alignment.TopEnd)  
            .padding(all = 4.dp)  
            .clip(shape = RoundedCornerShape(size = 4.dp))  
            .background(color = MaterialTheme.colorScheme.secondary)  
    ) {  
        Text(  
            text = product.category.name,  
            style = MaterialTheme.typography.labelSmall,  
            color = MaterialTheme.colorScheme.onSecondary,  
            modifier = Modifier.padding(horizontal = 6.dp, vertical = 2.dp)  
        )  
    }  
}
```

Gambar 14. Implementasi Image dan Box

5. Implementasi spacer dan Text

Silakan implementasikan kode seperti gambar 15 berikut, melanjutkan dari gambar 14 (**di luar Box**):

```
Spacer(modifier = Modifier.height(8.dp))

Text(
    text = product.name,
    style = MaterialTheme.typography.titleMedium,
    fontWeight = FontWeight.Bold,
    maxLines = 1,
    overflow = TextOverflow.Ellipsis
)

Spacer(modifier = Modifier.height(4.dp))

Text(
    text = "Rp ${product.price}",
    style = MaterialTheme.typography.bodyMedium,
    color = MaterialTheme.colorScheme.primary
)
```

Gambar 15. Implementasi Spacer dan Text

F. Membuat Layout untuk Menampilkan Kategori Produk

1. Deklarasi Function CategoryItem

Silakan implemmentasikan kode berikut:

```
@Composable
fun CategoryItem(category: Category, isSelected: Boolean, onClick: () -> Unit) {
```

Gambar 16. Deklarasi Function CategoryItem

Kode di atas merupakan deklarasi sebuah fungsi pembuat komponen antarmuka dalam *Jetpack Compose* yang ditandai dengan anotasi `@Composable`. Fungsi bernama **CategoryItem** dirancang untuk menampilkan satu buah komponen kategori dan menerima tiga buah parameter masukan. **Parameter pertama**, *category*, bertugas membawa informasi data kategori yang akan ditampilkan. Parameter kedua adalah *isSelected* yang bertipe Boolean (bernilai *true* atau *false*), parameter tersebut berguna sebagai **penanda status** untuk mengetahui apakah kategori

tersebut sedang aktif dipilih oleh pengguna atau tidak, sehingga sistem dapat memberikan efek visual yang berbeda, seperti mengubah warna latar belakangnya. Parameter `onClick` memanfaatkan fitur fungsi *Lambda* pada Kotlin (`() -> Unit`), yang memungkinkan fungsi ini menerima "sebuah aksi atau perintah" dari luar yang baru akan dieksekusi ketika pengguna mengetuk item kategori tersebut.

2. Implementasi Logic untuk Tampilan Kategori

Silakan tuliskan kode seperti gambar 17 di bawah ini:

```
@Composable
fun CategoryItem(category: Category, isSelected: Boolean, onClick: () -> Unit) {
    val containerColor = if (isSelected) MaterialTheme.colorScheme.primary else MaterialTheme.colorScheme.surfaceVariant
    val contentColor = if (isSelected) MaterialTheme.colorScheme.onPrimary else MaterialTheme.colorScheme.onSurfaceVariant

    Card(
        modifier = Modifier.clickable { onClick() },
        colors = CardDefaults.cardColors(
            containerColor = containerColor,
            contentColor = contentColor
        )
    ) {
        Text(
            text = category.name,
            modifier = Modifier.padding(horizontal = 16.dp, vertical = 8.dp),
            fontWeight = FontWeight.Medium
        )
    }
}
```

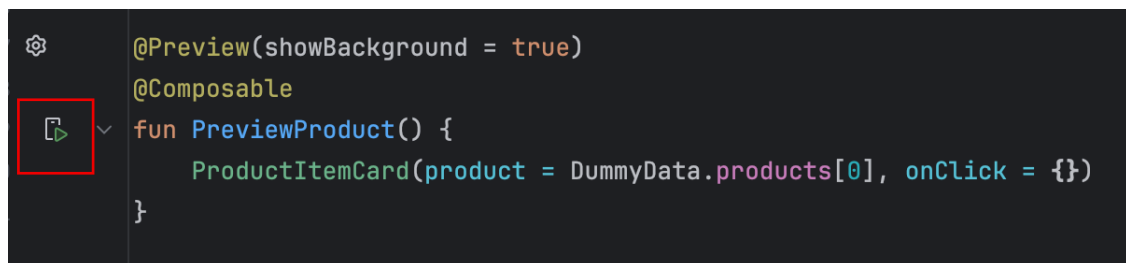
Gambar 17. Membuat Function untuk Menampilkan Kategori Produk

Kode di atas berfungsi untuk mengatur bagaimana sebuah komponen *Card* dapat berubah warna secara otomatis dan dapat berinteraksi saat di-tap/klik oleh pengguna. **Dua baris pertama** berfungsi untuk menentukan aturan kombinasi warna: **jika card tersebut sedang dalam keadaan dipilih (`isSelected`)**, maka sistem akan menerapkan warna utama aplikasi yang lebih mencolok untuk latar belakang maupun teksnya. Sebaliknya, jika tidak dipilih, kartu akan menggunakan warna dasar yang lebih pudar atau netral. Komponen kartu tersebut dibuat dan diberi kemampuan *clickable* agar ia dapat merespons serta menjalankan sebuah aksi ketika layarnya disentuh. Di bagian dalam kartu, disisipkan sebuah komponen teks (*Text*) yang berfungsi menampilkan nama kategori, di mana tampilannya diperindah dengan memberikan jarak ruang kosong di sekeliling teks (*padding*) dan sedikit penebalan huruf agar lebih rapi serta mudah dibaca.

G. Pratinjau Function dengan anotasi `@Preview`

Anotasi `@Preview` adalah fitur yang sangat memudahkan pengembang karena memungkinkan untuk melihat pratinjau desain antarmuka (UI) secara langsung di dalam panel Android Studio, tanpa harus menjalankan atau melakukan *build* aplikasi ke *emulator* atau perangkat fisik/*device*. Tambahan atribut `showBackground = true` berfungsi untuk memberikan latar belakang warna solid (*default*: putih) pada pratinjau, sehingga komponen UI lebih jelas terlihat. Di dalam fungsinya, komponen `ProductItemCard` dipanggil dengan disisipkan satu data contoh (`DummyData.products[0]`) dan sebuah aksi klik yang dibiarkan kosong (`onClick = {}`).

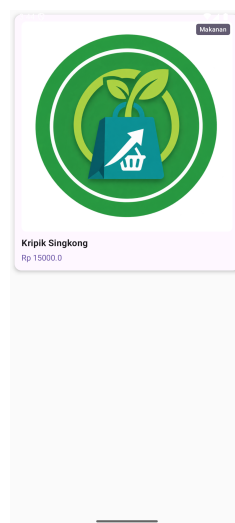
1. Silakan implementasikan kode berikut:



```
@Preview(showBackground = true)
@Composable
fun PreviewProduct() {
    ProductItemCard(product = DummyData.products[0], onClick = {})
}
```

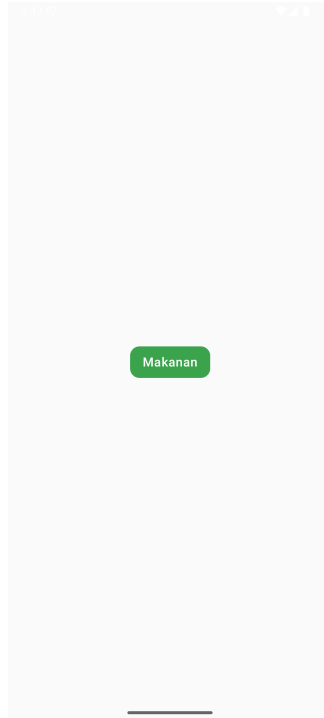
Gambar 18. Implementasi Function untuk Pratinjau

Kemudian jalankan *function* tersebut dengan klik *icon* seperti yang terdapat pada kotak warna merah di gambar 18. Hasil pratinjau akan muncul seperti pada gambar 19 berikut:



Gambar 19. Hasil Pratinjau

Buatkan pula function untuk preview kategori sehingga muncul tampilan seperti berikut (ICON HARUS BERADA DI TENGAH – TENGAH LAYAR) seperti gambar 20 berikut ini:



Gambar 20. Tampilan untuk Pratinjau Kategori

H. Implementasi Lazy List

1. Pada function **DaftarProdukScreen()** yang telah dibuat sebelumnya, silakan tambahkan kode untuk deklarasi variabel seperti pada gambar 21 berikut:

```
var selectedCategoryId by remember { mutableStateOf( value = DummyData.categories.firstOrNull()?.id) }

val filteredProducts = if (selectedCategoryId != null) {
    DummyData.products.filter { it.category_id == selectedCategoryId }
} else {
    DummyData.products
}

val context = LocalContext.current
val products = DummyData.products
```

Gambar 21. Deklarasi Variabel pada Function DaftarProdukScreen

Penjelasan Kode:

Immutable value adalah nilai atau variabel yang bersifat permanen dan sama sekali tidak dapat diubah lagi setelah pertama kali diberikan nilai awal. Dalam bahasa Kotlin, variabel tersebut dideklarasikan menggunakan kata kunci **val**. Kode pada gambar 21 berfungsi untuk menyimpan pilihan pengguna dan menyaring daftar produk berdasarkan kategori yang sedang aktif. Baris pertama menggunakan konsep inti Jetpack Compose yaitu *State* (`remember` dan `mutableStateOf`), yang bertugas untuk mengingat ID kategori yang dipilih, sehingga setiap kali pilihan tersebut diubah oleh pengguna, antarmuka layar akan secara otomatis diperbarui (*recomposition*) untuk merespons perubahan tersebut. Baris selanjutnya menggunakan keunggulan fitur *collection* pada Kotlin, yaitu fungsi `filter`, untuk menyaring data secara efisien. Sistem akan mengecek dan hanya memunculkan daftar produk yang sesuai dengan kategori yang dipilih, sedangkan jika tidak ada kategori yang aktif, maka seluruh daftar produk akan ditampilkan dari index pertama (*firstOrNull*).

perintah `firstOrNull` merupakan sebuah cara yang “aman” untuk mengambil data urutan paling awal dari sebuah daftar (*Collection*). Jika daftar tersebut memiliki isi, perintah tersebut akan mengambil item pertama. **Namun, jika ternyata daftar tersebut sama sekali tidak memiliki isi (kosong), sistem tidak akan memunculkan error yang dapat menyebabkan aplikasi mati mendadak (crash), melainkan hanya akan memberikan nilai kosong alias null.**

`val context = LocalContext.current`, digunakan untuk mengambil *Context* (konteks) dari lingkungan aplikasi Android saat ini. *Context* merupakan sebuah jembatan penghubung yang sangat penting dalam Android untuk mengakses sumber daya sistem atau melakukan aksi tertentu, seperti menampilkan pesan singkat (*Toast*), membuka halaman baru (*Intent*), atau mengakses sistem file. Baris kedua, `val products = DummyData.products`, berfungsi untuk mengambil data daftar produk yang telah kita buat di dalam objek *singleton* `DummyData` sebelumnya, lalu

menyimpannya ke dalam variabel lokal `products` agar namanya lebih singkat dan mudah saat hendak disalurkan ke komponen seperti `LazyColumn`.

1. Membuat `TopAppBar`

Di bawah deklarasi **Immutable Value**, implementasikan kode `TopAppBar` yang dapat menghasilkan tampilan sesuai gambar berikut **di dalam Scaffold** yang sudah dibuat pada `DaftarProductScreen.kt`:



Gambar 22. Implementasi `TopAppBar`

Peserta diwajibkan membuat tampilan seperti spesifikasi pada gambar 22 di atas.

2. Implementasi Kolom

Di bawah `AppBar`, yang telah dibuat pada tahapan sebelumnya. Silakan tambahkan kolom seperti gambar 23 berikut.

```
    ) { paddingValues ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(paddingValues)
        ) {
```

Gambar 23. Implementasi Kolom

3. Implementasi Tampilan Kategori

Silakan tuliskan kode sesuai pada gambar 24 berikut yang terletak di dalam **column**:

```
Text(
    text = "Kategori Produk",
    style = MaterialTheme.typography.titleLarge,
    modifier = Modifier.padding(all = 16.dp)
)

LazyRow(
    contentPadding = PaddingValues(horizontal = 16.dp),
    horizontalArrangement = Arrangement.spacedBy(8.dp)
) {
    items(DummyData.categories) { category ->
        CategoryItem(
            category = category,
            isSelected = category.id == selectedCategoryId,
            onClick = { selectedCategoryId = category.id }
        )
    }
}

Spacer(modifier = Modifier.height(16.dp))
Text(
    text = "Daftar Produk",
    style = MaterialTheme.typography.titleLarge,
    modifier = Modifier.padding(horizontal = 16.dp, vertical = 8.dp)
)
```

Gambar 24. Implementasi Tampilan Kategori

LazyRow merupakan inti dalam Jetpack Compose yang berfungsi untuk menampilkan daftar data secara mendatar (horizontal) yang bisa digeser ke samping. Sifatnya yang "malas" (*lazy*) membuat hemat dalam penggunaan memori, karena sistem hanya akan memproses dan menampilkan item-item yang sedang terlihat di layar pengguna saja. Pengaturan `contentPadding` dan `horizontalArrangement` ditambahkan untuk memberikan jarak estetika, baik berupa ruang kosong di sisi ujung layar maupun celah spasi di antara masing-masing kategori agar tidak saling berdempetan.

Di bagian dalamnya, Kotlin menggunakan perintah *items* untuk secara otomatis melakukan **perulangan pada seluruh data kategori**. Sistem akan menampilkan komponen `CategoryItem` di layar yaitu dengan cara cek apakah kategori tersebut adalah kategori yang sedang aktif dipilih saat ini

(isSelected), dan jika pengguna melakukan *tap*/klik kategori tersebut (onClick), sistem akan langsung memperbarui status pilihan (selectedCategoryId), yang memicu antarmuka layar untuk berubah mengikuti pilihan terbaru.

4. Implementasi Tampilan Daftar Produk Berdasarkan Kategori

Silakan implementasikan kode berikut:

```
LazyVerticalGrid(  
    columns = GridCells.Fixed( count = 2),  
    contentPadding = PaddingValues( all = 16.dp),  
    horizontalArrangement = Arrangement.spacedBy(16.dp),  
    verticalArrangement = Arrangement.spacedBy(16.dp),  
    modifier = Modifier.fillMaxSize()  
) {  
  
    items(filteredProducts) { product ->  
        ProductItemCard(product = product) {  
            Toast.makeText(context, text = "Clicked: ${product.name}",  
                duration = Toast.LENGTH_SHORT)  
                .show()  
        }  
    }  
}
```

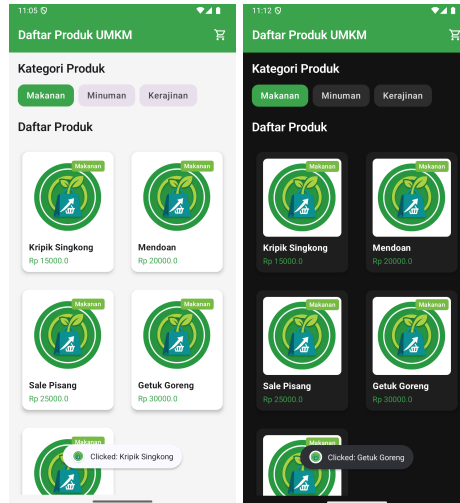
Gambar 25. Implementasi Tampilan Daftar Produk Berdasarkan Kategori

Kode di atas menggunakan komponen `LazyVerticalGrid` pada Jetpack Compose untuk menyusun dan menampilkan daftar produk ke dalam bentuk kisi-kisi atau kotak-kotak (*grid*) dengan format tepat dua kolom secara vertikal. Sama seperti tata letak "malas" (*lazy*) lainnya di Compose, komponen ini bekerja dengan sangat efisien karena hanya memuat kartu produk yang sedang benar-benar terlihat di layar ponsel. Pengaturan tambahan seperti `contentPadding` dan `Arrangement.spacedBy` berfungsi untuk memberikan jarak spasi yang seragam dan rapi, baik di batas tepi luar layar maupun di celah antar kartu produk agar tidak saling

berdempetan, sementara `Modifier.fillMaxSize()` memastikan kisi-kisi tersebut merentang penuh menutupi seluruh ruang layar yang tersedia. Di bagian dalam blok tersebut, Kotlin secara otomatis melakukan perulangan pada daftar `filteredProducts` (produk yang sudah disaring) menggunakan perintah `items`. Untuk setiap produk yang ada, sistem akan menggambar sebuah `ProductItemCard` di layar.

Saat pengguna mengeklik salah satu *card* produk, aksi tersebut akan memicu pemanggilan **Toast**. **Toast adalah komponen bawaan Android berupa pesan notifikasi kecil semi-transparan yang muncul sekilas di layar (biasanya di bagian bawah), lalu menghilang secara otomatis setelah beberapa saat. Fungsi utama Toast adalah untuk memberikan umpan balik (feedback) informasi yang cepat kepada pengguna tanpa menghalangi antarmuka atau mengganggu aktivitas mereka.** Saat *card* di-klik, *Toast* akan memberikan konfirmasi dengan memunculkan teks bertuliskan "**Clicked:** " diikuti dengan nama produk yang bersangkutan.

5. Buatlah Function Preview untuk menampilkan hasil dari tahapan di atas. Hasil dari Preview menampilkan hasil DARK & LIGHT THEME

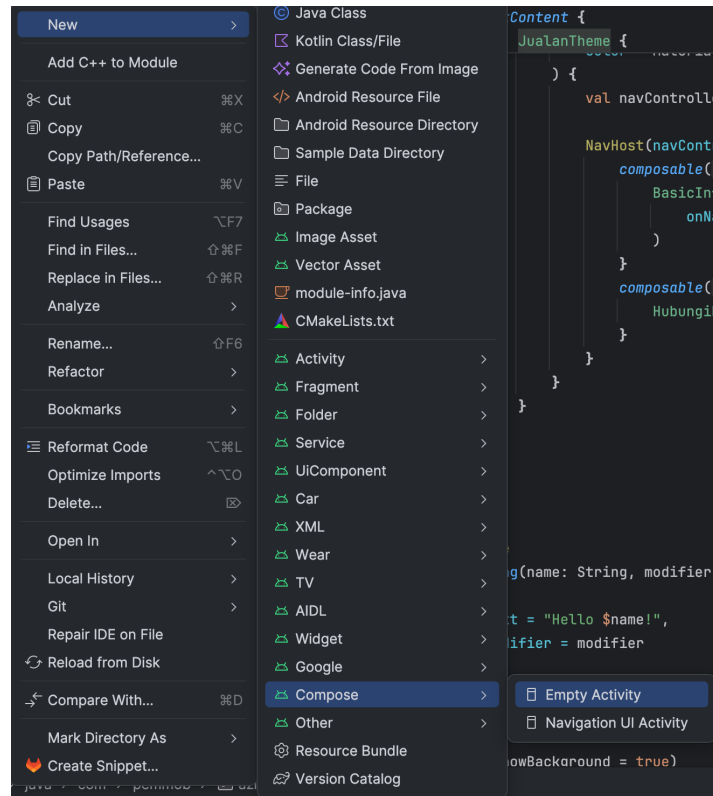


Gambar 26. Tampilan Preview Dark & Light

I. Membuat Activity Baru

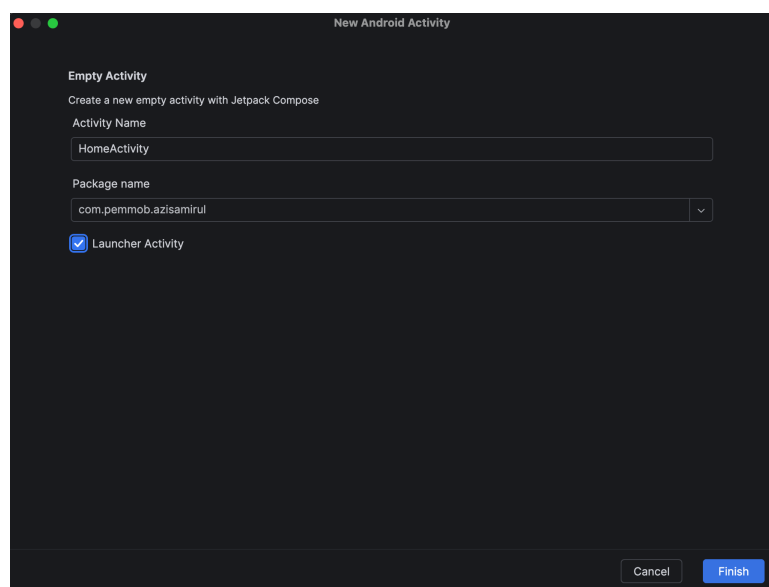
Pada bagian ini, kita akan membuat *activity* baru. **Activity** dapat dianalogikan sebagai satu layar antarmuka tunggal tempat pengguna dapat berinteraksi. Sama halnya seperti sebuah jendela browser atau satu halaman buku, setiap Activity bertanggung jawab untuk menampilkan tata letak visual (UI) tertentu dan merespons segala tindakan pengguna di layar tersebut, seperti ketukan tombol, input teks, atau gestur. Dalam sebagian besar aplikasi, misalnya saat berpindah dari beranda ke halaman profil atau masuk ke layar rincian barang, pengguna sebenarnya sedang beralih dari satu Activity ke Activity lainnya yang telah disiapkan oleh pengembang.

2. Untuk memulai pembuatan activity, silakan klik kanan pada direktori/folder App
→ pilih **New** → **Compose** → **Empty Activity**.



Gambar 27. Menu untuk Membuat Activity Baru

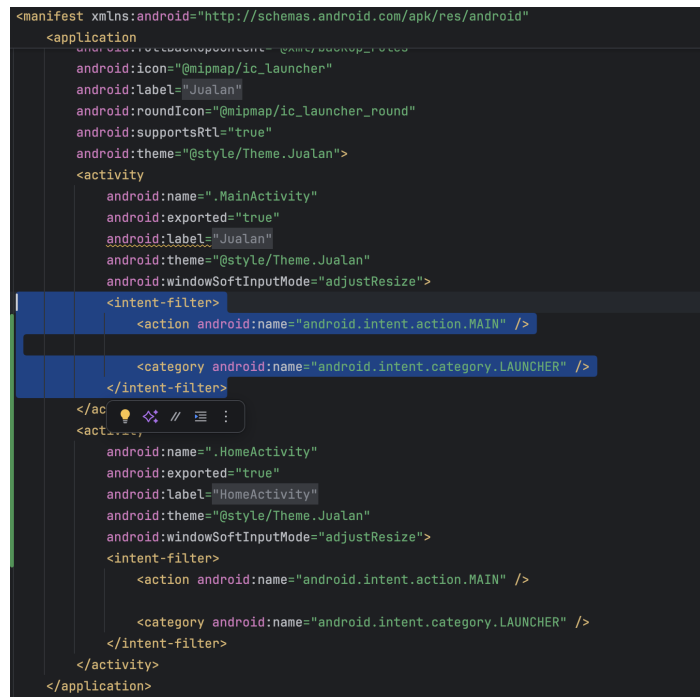
3. Kemudian Isikan activity name dengan nama HomeActivity dan centang bagian *Launcher Activity*. Kemudian klik *Finish* dan tunggu beberapa saat.



Gambar 28. Pengisian Kolom untuk Membuat Activity

Launcher Activity adalah layar atau halaman pertama yang secara otomatis terbuka ketika pengguna mengetuk ikon aplikasi di menu utama ponsel. Agar sistem operasi Android mengetahui layar mana yang harus dimunculkan pertama kali, pengembang harus mendeklarasikannya di dalam sebuah file konfigurasi sentral yang bernama **AndroidManifest.xml**. Di dalam file *manifest* tersebut, salah satu *Activity* akan diberi kode penanda khusus (berupa *Intent Filter* dengan kategori LAUNCHER dan *action* MAIN), yang berfungsi sebagai instruksi bagi sistem Android bahwa layar tersebut adalah pintu masuk utama untuk memulai aplikasi.

4. Silakan buka **AndroidManifest.xml**, kemudian cek di blok **Activity** → **.MainActivity**, kemudian hapus bagian kode yang terdapat di blok tersebut seperti pada gambar 29 yang diblok warna biru. Hal ini bertujuan agar tidak terjadi duplikasi untuk deklarasi *Launcher* pada aplikasi yang kita buat.



```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    android:versionCode="1"
    android:versionName="1.0"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportRtl="true"
    android:theme="@style/Theme.Jualan">
    <activity
        android:name=".MainActivity"
        android:exported="true"
        android:label="@string/app_name"
        android:theme="@style/Theme.Jualan"
        android:windowSoftInputMode="adjustResize">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
    <activity
        android:name=".HomeActivity"
        android:exported="true"
        android:label="@string/app_name"
        android:theme="@style/Theme.Jualan"
        android:windowSoftInputMode="adjustResize">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</manifest>
```

Gambar 29. Penyesuaian Deklarasi Launcher pada *AndroidManifest*

J. Modifikasi HomeActivity

Silakan buka **HomeActivity.kt** kemudian ubah baris kode pada function **onCreate** seperti pada gambar 30 berikut ini dan hapus *function* lain karena tidak digunakan.

```
Usage
class HomeActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            JualanTheme {
                DaftarProdukScreen()
            }
        }
    }
}
```

Gambar 30. Modifikasi HomeActivity

JALANKAN APLIKASI TERSEBUT DI EMULATOR/PERANGKAT ANDA